

Laravel vs React.js: Evaluating Core Web Vitals Across Server-Side and Client-Side Rendering using Google Lighthouse

Farah Aulia Zahra
Informatika
Universitas Diponegoro
Semarang, Indonesia
farahauliazahra@students.undip.ac.id

Elvina Neila Samas
Informatika
Universitas Diponegoro
Semarang, Indonesia
elvinaneila@students.undip.ac.id

Febrianti Pujiatiningsih
Informatika
Universitas Diponegoro
Semarang, Indonesia
febriantipuji@students.undip.ac.id

Abstract—Performa web merupakan faktor krusial dalam pengembangan aplikasi modern yang berpengaruh langsung terhadap pengalaman pengguna dan kualitas layanan digital. Penelitian ini bertujuan membandingkan performa *Core Web Vitals* antara *Server-Side Rendering* (SSR) pada Laravel dan *Client-Side Rendering* (CSR) pada React.js menggunakan Google Lighthouse sebagai alat ukur. Pengujian dilakukan pada dua aplikasi web resep masakan dengan konten identik, mencakup halaman Home dan halaman Detail Resep, masing-masing diuji sebanyak lima kali. Hasil penelitian menunjukkan bahwa Laravel SSR secara konsisten memberikan performa pemuatan awal yang lebih baik dibandingkan React.js CSR, sehingga lebih direkomendasikan untuk *website* berbasis konten dengan tingkat interaksi yang terbatas.

Keywords—*Core Web Vitals*, *Server-Side Rendering*, *Client-Side Rendering*, *Laravel*, *React.js*, *Google Lighthouse*, *Performa Web*

I. INTRODUCTION (HEADING 1)

Aspek performa pada halaman web kini menempati posisi yang semakin strategis dalam proses pengembangan aplikasi web modern, baik dari perspektif pengguna maupun pengembang. Optimasi performa web merupakan aspek krusial dalam ekosistem digital modern yang secara langsung memengaruhi kepuasan pengguna, tingkat keterlibatan, serta keberhasilan bisnis secara keseluruhan [1]. Halaman web yang memerlukan waktu muat panjang, lamban merespons interaksi, maupun menampilkan ketidakstabilan elemen visual dapat menurunkan kenyamanan pengguna, yang pada akhirnya tercermin pada tingginya *bounce rate* dan melemahnya tingkat konversi. Dengan demikian, performa web telah berkembang dari sekadar urusan teknis menjadi elemen strategis yang tidak terpisahkan dari pengembangan produk digital.

Perkembangan ini semakin diperkuat melalui kebijakan Google pada tahun 2020 yang secara resmi memasukkan *Core Web Vitals* (CWV) ke dalam algoritma pemeringkatan SEO-nya. Seiring dengan meningkatnya kompleksitas aplikasi web, Google memperkenalkan Web Vitals sebagai seperangkat metrik terstandarisasi yang dirancang untuk mengukur kualitas pengalaman pengguna dari sisi performa pemuatan, responsivitas interaksi, maupun stabilitas visual halaman [1]. Metrik utama CWV meliputi *Largest Contentful Paint* (LCP), dan *Cumulative Layout Shift* (CLS). Metrik komplementer seperti *First Contentful Paint* (FCP), dan *Total Blocking Time* (TBT) turut memberikan gambaran performa yang lebih menyeluruh.

Dalam membangun web yang cepat, salah satu keputusan paling krusial adalah memilih metode *rendering*. Dua metode yang paling umum dipakai saat ini adalah *Server-Side Rendering* (SSR) dan *Client-Side Rendering* (CSR). Pada

metode SSR (misalnya menggunakan *framework* Laravel), proses pembentukan kode HTML diselesaikan di server. Alhasil, browser pengguna tinggal menampilkan halaman yang sudah jadi tanpa harus menjalankan JavaScript yang berat terlebih dahulu. Sebaliknya, metode CSR (seperti pada React.js) menyerahkan proses *rendering* ke browser pengguna. Browser harus mengunduh dan menjalankan kumpulan file JavaScript terlebih dahulu sebelum antarmuka halaman bisa dibangun dan ditampilkan. Lokhande et al. [2] telah membandingkan SSR dan CSR secara konseptual beserta *framework-framework* yang umum digunakan, namun penelitian tersebut belum menyertakan pengukuran empiris berbasis metrik performa yang terstandarisasi. Penelitian ini hadir untuk mengisi celah tersebut dengan melakukan pengujian langsung pada dua implementasi identik, satu berbasis Laravel (SSR) dan satu berbasis React.js (CSR), dengan menggunakan Google Lighthouse sebagai alat ukur *Core Web Vitals* secara objektif dan terukur.

Wehner et al. [3] menemukan bahwa *Core Web Vitals*, khususnya LCP, berkorelasi dengan kualitas pengalaman pengguna, meskipun *loading time* tetap menjadi faktor dominan. Namun, studi tersebut belum mengkaji secara langsung bagaimana perbedaan arsitektur *rendering* berkontribusi terhadap perbedaan skor CWV yang dihasilkan. Kondisi ini mendorong banyak pengembang untuk memilih metode *rendering* berdasarkan kebiasaan atau preferensi pribadi, bukan berdasarkan bukti kuantitatif yang terukur.

Berdasarkan celah (*gap*) penelitian tersebut, penelitian ini dilakukan untuk membandingkan secara langsung performa *Core Web Vitals* antara implementasi SSR pada Laravel dan CSR pada React.js. Pengujian ini menggunakan aplikasi web dengan konten yang sama persis dan diukur menggunakan instrumen pengukuran standar, yaitu Google Lighthouse. Penelitian ini diharapkan bisa memberikan hasil yang objektif dan berbasis data, sehingga dapat menjadi panduan praktis bagi pengembang dalam memilih arsitektur *rendering* yang paling sesuai untuk kebutuhan proyek mereka.

II. TINJAUAN PUSTAKA

A. Performa Web

Performa web merupakan aspek penting dalam pengembangan *website* karena berpengaruh langsung terhadap pengalaman pengguna, kepuasan, dan kualitas layanan digital [1]. *Website* dengan waktu muat yang lambat dapat menurunkan kenyamanan pengguna serta mengurangi interaksi pengguna terhadap sistem [4]. Selain itu, optimasi performa *website* secara berkelanjutan dapat meningkatkan kualitas pengalaman pengguna dan efektivitas akses informasi pada aplikasi berbasis web modern.

Menurut Jain [1], performa web yang baik berkontribusi langsung terhadap peningkatan kualitas layanan digital dan pengalaman pengguna dalam mengakses aplikasi berbasis web. Selain itu, van Riet, Malavolta, dan Ghaleb [4] menyatakan bahwa optimasi performa web secara berkelanjutan penting dilakukan untuk meningkatkan efisiensi sistem serta menjaga kualitas interaksi pengguna terhadap *website*.

B. Core Web Vitals (CWV)

Core Web Vitals (CWV) merupakan standar pengukuran performa *website* yang digunakan untuk mengevaluasi pengalaman pengguna berdasarkan kecepatan pemuatan halaman, respons interaksi, dan kestabilan tampilan visual [1]. Pengukuran ini membantu pengembang dalam mengidentifikasi masalah performa yang dapat memengaruhi kenyamanan pengguna saat mengakses *website*. Tiga indikator utama dalam *Core Web Vitals* terdiri dari *Largest Contentful Paint* (LCP), *First Input Delay* (FID), dan *Cumulative Layout Shift* (CLS) yang digunakan untuk mengukur performa pemuatan konten, responsivitas, dan stabilitas visual halaman web [1].

Wehner dkk. [3] menyatakan bahwa *Largest Contentful Paint* (LCP) memiliki korelasi positif terhadap kualitas pengalaman pengguna (*Quality of Experience/QoE*), meskipun waktu *loading* secara keseluruhan tetap menjadi faktor yang paling dominan dalam menentukan kualitas pengalaman pengguna saat mengakses *website*. Selain itu, Wehner dkk. [5] juga menjelaskan bahwa metrik *Core Web Vitals* dapat digunakan sebagai indikator evaluasi performa *website modern*, namun hasil pengukurannya tetap perlu dipertimbangkan bersama faktor pengalaman pengguna secara nyata.

C. Server-Side Rendering (SSR)

Server-Side Rendering (SSR) merupakan metode rendering halaman web yang proses pembentukan tampilannya dilakukan di sisi server sebelum dikirim ke browser dalam bentuk dokumen HTML lengkap. Pendekatan ini memungkinkan halaman dapat ditampilkan lebih cepat karena browser tidak perlu melakukan *rendering* awal menggunakan JavaScript secara penuh [6].

Beberapa penelitian menunjukkan bahwa pendekatan SSR memiliki performa *loading* awal yang lebih baik dibandingkan CSR. Iskandar et al. [7] menyatakan bahwa situs web berbasis SSR menghasilkan waktu *First Contentful Paint* (FCP) yang lebih cepat karena proses rendering dilakukan di sisi server sebelum halaman dikirim ke browser. Gangishetti dan Jain [8] juga menunjukkan hasil yang serupa, bahwa SSR cenderung menghasilkan nilai *Largest Contentful Paint* (LCP) dan *First Contentful Paint* (FCP) yang lebih baik dibandingkan implementasi CSR yang setara, terutama pada jaringan lambat atau perangkat dengan spesifikasi rendah. Selain itu, pendekatan SSR juga mempermudah proses *indexing* oleh mesin pencari karena konten halaman telah tersedia sejak awal halaman dimuat [7].

D. Client-Side Rendering (CSR)

Client-Side Rendering (CSR) merupakan pendekatan rendering halaman web yang proses pembentukan tampilannya dilakukan di sisi klien menggunakan JavaScript. Pada pendekatan ini, browser akan memproses dan

membangun tampilan halaman setelah menerima file JavaScript dari server [9].

Beberapa penelitian menunjukkan bahwa CSR memiliki keunggulan dalam interaktivitas dan responsivitas aplikasi karena perubahan tampilan dapat dilakukan tanpa memuat ulang seluruh halaman. Iskandar et al. [7] menyatakan bahwa pendekatan CSR lebih sesuai digunakan pada aplikasi web dinamis yang membutuhkan interaksi pengguna secara *real-time*. Selain itu, Gangishetti dan Jain [8] menjelaskan bahwa CSR memungkinkan proses navigasi halaman berlangsung lebih fleksibel setelah proses *rendering* awal selesai dilakukan.

Namun demikian, pendekatan CSR memiliki kelemahan pada performa pemuatan awal halaman karena browser harus memproses file JavaScript terlebih dahulu sebelum konten dapat ditampilkan secara penuh. Kondisi tersebut menyebabkan nilai *First Contentful Paint* (FCP) dan *Largest Contentful Paint* (LCP) pada CSR cenderung lebih lambat dibandingkan SSR, terutama pada perangkat dengan spesifikasi rendah atau jaringan lambat [7], [8].

E. Framework Laravel

Laravel adalah *framework* PHP populer untuk pengembangan aplikasi web modern yang menerapkan arsitektur MVC (*Model-View-Controller*) [2]. Secara *default*, Laravel menggunakan Blade sebagai *template engine* yang mendukung *Server-Side Rendering* (SSR), memungkinkan penyisipan data *backend* langsung ke struktur HTML sebelum dikirim ke browser [10].

Fahrudin et al. (2025) menunjukkan bahwa Blade unggul dalam kecepatan *loading* awal dan kompatibilitas SEO, namun mengalami keterbatasan signifikan dalam skalabilitas pada skenario data besar (10.000 entri), Total Blocking Time (TBT) melonjak drastis hingga 2.790 ms, melampaui ambang toleransi pengguna 10 detik yang direkomendasikan Jakob Nielsen [10]. Temuan ini mengindikasikan bahwa meskipun Blade efisien untuk aplikasi berbasis konten statis, pendekatan ini kurang memadai untuk sistem yang membutuhkan interaktivitas tinggi dengan volume data besar.

F. Library React.js

React.js adalah *library* JavaScript yang dikembangkan oleh Facebook (sekarang Meta) untuk membangun antarmuka pengguna interaktif dan dinamis melalui pendekatan *Client-Side Rendering* (CSR) [2]. Dalam CSR, browser menerima HTML *skeleton* minimal beserta file JavaScript, kemudian *framework* seperti React membangun tampilan secara dinamis di sisi klien menggunakan virtual DOM [10].

Keunggulan utama React terletak pada stabilitas interaktivitasnya pada skala data besar. Penelitian Fahrudin et al. (2025) menunjukkan bahwa React hanya mencatat TBT sebesar 70 ms pada 10.000 entri, jauh lebih rendah dibandingkan Blade yang mencapai 2.790 ms pada volume data yang sama [10]. Meskipun React memiliki kelemahan berupa waktu *loading* awal yang lebih lambat, keunggulan responsivitas *post-load* menjadikannya pilihan yang lebih tepat untuk aplikasi berbasis data dengan kebutuhan interaktivitas tinggi [10].

G. Google Lighthouse

Google Lighthouse adalah alat audit otomatis untuk mengevaluasi kualitas halaman web berdasarkan metrik *Core Web Vitals* yang terdiri dari *Largest Contentful Paint (LCP)*, *First Input Delay (FID)*, dan *Cumulative Layout Shift (CLS)* [3].

Menurut McGill et al. (2023), Lighthouse digunakan sebagai *tools open-source* untuk pengukuran performa dan audit aksesibilitas guna meningkatkan kualitas *website*, namun memiliki keterbatasan dalam menangani *input* berganda dan *format output* laporan yang terbatas [11]. Penelitian Wehner et al. (2022) juga menemukan bahwa *Core Web Vitals* memiliki nilai prediktif yang lebih rendah terhadap *Quality of Experience (QoE)* pengguna dibandingkan waktu pemuatan halaman dan *Speed Index* tradisional [3].

III. METODOLOGI

Penelitian ini menggunakan eksperimen kuantitatif dengan pendekatan komparatif. Fokus penelitian adalah membandingkan performa *Core Web Vitals* pada dua aplikasi web resep masakan yang memiliki isi dan fungsi yang sama, tetapi menggunakan metode *rendering* berbeda. Aplikasi pertama dibuat dengan Laravel menggunakan *Server-Side Rendering (SSR)*, sedangkan aplikasi kedua dibuat dengan React.js menggunakan *Client-Side Rendering (CSR)*. Performa kedua aplikasi diuji menggunakan Google Lighthouse.

A. Desain Penelitian

Pengujian dilakukan secara langsung pada dua aplikasi web yang telah dibuat. Kedua aplikasi dirancang dengan konten, struktur halaman, dan tampilan yang serupa agar kondisi pengujiannya tetap konsisten. Perbedaan utama dari kedua aplikasi hanya terdapat pada teknologi dan metode *rendering* yang digunakan.

Halaman yang diuji terdiri dari halaman Home dan halaman Detail Resep. Halaman Home menampilkan kumpulan resep masakan dalam bentuk kartu atau daftar resep. Sementara itu, halaman Detail Resep berisi informasi lengkap dari salah satu resep, seperti gambar makanan, nama resep, bahan-bahan, dan langkah pembuatannya.

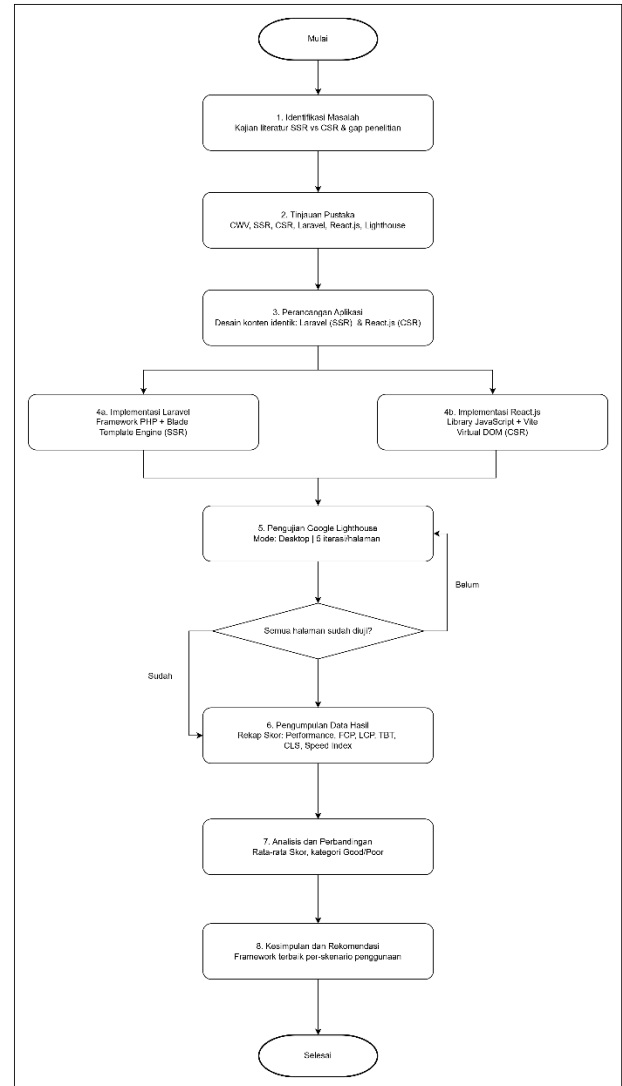
Dua halaman tersebut dipilih karena mewakili pola umum pada website berbasis konten, yaitu halaman daftar dan halaman detail. Dengan struktur halaman yang sama, perbedaan hasil pengujian dapat lebih diarahkan pada perbedaan performa antara SSR pada Laravel dan CSR pada React.js.

B. Research Flowchart

Alur penelitian ditunjukkan pada Gambar 1. Tahapan penelitian dimulai dari identifikasi masalah, kemudian dilanjutkan dengan tinjauan pustaka mengenai *Core Web Vitals*, SSR, CSR, Laravel, React.js, dan Google Lighthouse.

Tahap berikutnya adalah perancangan aplikasi resep masakan. Aplikasi dibuat dalam dua versi, yaitu Laravel SSR dan React.js CSR. Setelah kedua aplikasi selesai dibuat, masing-masing halaman diuji menggunakan Google Lighthouse. Data hasil pengujian kemudian dikumpulkan, dibandingkan, dan digunakan untuk menarik kesimpulan.

Gambar 1. Research Flowchart



C. Instrument / Tool

Instrumen utama yang digunakan dalam penelitian ini adalah Google Lighthouse. Alat ini digunakan untuk mengukur performa website berdasarkan metrik *Core Web Vitals* dan beberapa metrik pendukung, seperti *Performance Score*, FCP, LCP, TBT, CLS, dan *Speed Index*. Pengujian dilakukan melalui Google Chrome pada *website* yang telah di-deploy.

Selain itu, penelitian ini menggunakan Laravel, React.js, Vercel, dan spreadsheet. Laravel digunakan untuk membuat versi *website* dengan pendekatan *Server-Side Rendering (SSR)*, sedangkan React.js digunakan untuk membuat versi *website* dengan pendekatan *Client-Side Rendering (CSR)*. Vercel digunakan sebagai platform *deployment* agar kedua *website* dapat diuji melalui URL *online*. Spreadsheet digunakan untuk mencatat hasil pengujian dan menghitung nilai rata-rata dari setiap metrik.

D. Population and Sampling Technique

Populasi dalam penelitian ini adalah *website* yang menggunakan metode *rendering* berbeda dan sampel yang digunakan berupa dua *website* resep masakan yang dibuat

khusus untuk pengujian, yaitu *website* Laravel SSR dan *website* React.js CSR.

Teknik *sampling* yang digunakan adalah *purposive sampling*, karena sampel dipilih berdasarkan kebutuhan penelitian, bukan secara acak. Kedua *website* dibuat dengan tema, konten, gambar, struktur halaman, dan tampilan yang sama. Perbedaan utama hanya terletak pada metode *rendering* yang digunakan.

E. Data Collection Technique / Strategy

Data dikumpulkan melalui pengujian secara langsung menggunakan Google Lighthouse pada dua halaman, yaitu halaman Home dan halaman Detail Resep. Pengujian dilakukan pada kedua implementasi *website*, yaitu Laravel SSR dan React.js CSR.

Setiap halaman diuji beberapa kali agar hasilnya tidak terlalu dipengaruhi oleh kondisi sesaat pada perangkat atau browser. Hasil dari setiap pengujian kemudian dicatat dalam *spreadsheet*, lalu dihitung nilai rata-ratanya untuk digunakan pada tahap analisis.

Table 1. Jadwal *Data Collection*

Tahap	Aktivitas	Tools/Teknologi	Output
1	Perencanaan struktur dan fitur <i>website</i> resep masakan	Figma, Draw io	Desain <i>Flowchart</i> , <i>website</i>
2	Pembuatan <i>website</i> berbasis Laravel dengan metode SSR	Laravel, PHP, Blade	<i>Website</i> Laravel SSR
3	Pembuatan <i>website</i> berbasis React.js dengan metode CSR	React.js, JavaScript, CSS	<i>Website</i> React.js CSR
4	Implementasi halaman Home dan Detail Resep pada kedua <i>website</i>	Laravel dan React.js	Halaman pengujian
5	<i>Deployment Website</i>	Vercel	URL <i>website</i> untuk pengujian
6	Pengujian performa menggunakan Google Lighthouse	Google Chrome, Lighthouse	Data <i>Core Web Vitals</i>
7	Pengulangan pengujian sebanyak 5 kali pada setiap halaman	Google Lighthouse	Data pengujian stabil
8	Pencatatan hasil pengujian ke <i>spreadsheet</i>	<i>Spreadsheet</i>	<i>Dataset</i> penelitian
9	Perhitungan rata-rata dan analisis data	<i>Spreadsheet</i>	Hasil analisis performa

F. Data Analysis Technique

Data dianalisis menggunakan teknik deskriptif komparatif. Hasil pengujian dari Laravel SSR dan React.js CSR dibandingkan berdasarkan nilai rata-rata dari setiap metrik.

Metrik yang digunakan dalam penelitian ini meliputi:

- *Performance Score* : skor keseluruhan performa *website* dari Google Lighthouse.
- *Largest Contentful Paint* (LCP) : waktu munculnya konten terbesar pada halaman.
- *Cumulative Layout Shift* (CLS) : tingkat kestabilan tampilan visual halaman.
- *First Contentful Paint* (FCP) : waktu munculnya konten pertama pada halaman.
- *Total Blocking Time* (TBT) : total waktu browser tidak responsif terhadap input pengguna.
- *Speed Index* : kecepatan halaman dalam menampilkan konten visual.

Pada *Performance Score*, nilai yang lebih tinggi menunjukkan performa yang lebih baik. Sementara itu, pada metrik LCP, CLS, FCP, TBT, dan *Speed Index*, nilai yang lebih rendah menunjukkan performa yang lebih baik.

Hasil analisis digunakan untuk melihat metode *rendering* mana yang memberikan performa lebih baik pada *website* resep masakan.

IV. HASIL DAN PEMBAHASAN

Pengujian dilakukan menggunakan Google Lighthouse pada dua halaman, yaitu halaman Home dan halaman Detail Resep. Setiap halaman diuji sebanyak lima kali pada dua implementasi *website*, yaitu Laravel SSR dan React.js CSR. Nilai yang digunakan dalam pembahasan adalah nilai rata-rata dari lima kali pengujian. Metrik yang dianalisis meliputi *Performance Score*, FCP, LCP, TBT, CLS, dan *Speed Index*.

A. Hasil Pengujian Halaman Home

Pengujian pertama dilakukan pada halaman Home. Halaman ini berisi daftar resep dalam bentuk *card* yang menampilkan gambar, judul resep, deskripsi singkat, dan tombol untuk melihat detail resep.

Gambar 2. Hasil Pengujian Laravel SSR pada Halaman Home

	Pengujian	Performance Score	FCP (s)	LCP (s)	TBT (ms)	CLS	Speed Index
Home Laravel	1	98	0,7	1,0	0	0	0,7
	2	96	1,0	1,2	0	0	1,0
	3	96	1,1	1,2	0	0	1,1
	4	96	1,1	1,1	0	0	1,1
	5	96	1,0	1,1	0	0	1
		96,4	0,98	1,1	0	0	0,98

Gambar 3. Hasil Pengujian React CSR pada Halaman Home

	Pengujian	Performance Score	FCP (s)	LCP (s)	TBT (ms)	CLS	Speed Index
Home React	1	97	0,9	0,9	0	0,002	1,3
	2	94	1,1	1,1	0	0,002	1,5
	3	84	0,5	0,7	360	0,002	1
	4	83	0,8	0,8	360	0,002	1,2
	5	88	0,7	0,8	270	0,002	1,1
		89,2	0,8	0,86	198	0,002	1,22

Berdasarkan hasil pengujian pada Gambar 2, implementasi Laravel SSR pada halaman Home memperoleh

rata-rata *Performance Score* sebesar 96,4, dengan nilai FCP sebesar 0,98 detik, LCP sebesar 1,1 detik, TBT sebesar 0 ms, CLS sebesar 0, dan *Speed Index* sebesar 0,98. Hasil tersebut menunjukkan bahwa halaman Home berbasis Laravel SSR mampu dimuat dengan cepat karena proses rendering telah dilakukan di sisi server sebelum halaman dikirimkan ke browser.

Sementara itu, berdasarkan hasil pengujian pada Gambar 3, implementasi React.js CSR pada halaman Home memperoleh rata-rata *Performance Score* sebesar 89,2, dengan nilai FCP sebesar 0,8 detik, LCP sebesar 0,86 detik, TBT sebesar 198 ms, CLS sebesar 0,002, dan *Speed Index* sebesar 1,22. Hasil ini menunjukkan bahwa proses *rendering* di sisi klien menyebabkan waktu pemuatan halaman menjadi lebih lama karena browser harus mengunduh dan menjalankan file JavaScript terlebih dahulu sebelum seluruh konten dapat ditampilkan secara penuh.

Perbandingan kedua implementasi pada halaman Home menunjukkan bahwa Laravel SSR memiliki performa pemuatan awal yang lebih baik dibandingkan React.js CSR. Perbedaan paling signifikan terlihat pada nilai LCP dan *Speed Index*, di mana Laravel SSR mampu menampilkan konten utama halaman lebih cepat dibandingkan React.js CSR karena proses rendering dilakukan langsung di server.

B. Hasil Pengujian Halaman Detail Resep

Pengujian kedua dilakukan pada halaman Detail Resep. Halaman ini berisi gambar resep, nama resep, deskripsi, bahan-bahan, dan langkah pembuatan. Halaman Detail Resep diuji karena mewakili halaman konten utama yang lebih lengkap dibandingkan halaman Home.

Gambar 4. Hasil Pengujian Laravel SSR pada Halaman Detail

	Pengujian	Performance Score	FCP (s)	LCP (s)	TBT (ms)	CLS	Speed Index
Detail Laravel	1	96	1,1	1,1	0	0	1,1
	2	97	1,0	1,0	0	0	1,0
	3	97	1,0	1,0	0	0	1,0
	4	99	0,7	0,8	0	0	0,7
	5	99	0,7	0,8	0	0	0,7
		97,6	0,9	0,94	0	0	0,9

Gambar 5. Hasil Pengujian React CSR pada Halaman Detail

	Pengujian	Performance Score	FCP (s)	LCP (s)	TBT (ms)	CLS	Speed Index
Detail React	1	99	0,7	0,8	20	0,002	0,8
	2	92	0,6	0,9	10	0	2,9
	3	85	0,6	0,7	290	0,002	1,6
	4	94	0,7	0,7	0	0,002	0,7
	5	90	0,5	0,5	50	0	0,8
		92	0,62	0,72	74	0,0012	1,36

Berdasarkan hasil pengujian pada Gambar 4, implementasi Laravel SSR pada halaman Detail Resep memperoleh rata-rata *Performance Score* sebesar 97,6, dengan nilai FCP sebesar 0,9 detik, LCP sebesar 0,94 detik, TBT sebesar 0 ms, CLS sebesar 0, dan *Speed Index* sebesar 0,9. Hasil tersebut menunjukkan bahwa Laravel SSR mampu mempertahankan performa pemuatan halaman dengan baik meskipun halaman memuat konten yang lebih lengkap dibandingkan halaman Home, karena proses rendering dilakukan di sisi server sebelum halaman dikirimkan ke browser.

Sementara itu, berdasarkan hasil pengujian pada Gambar 5, implementasi React.js CSR pada halaman Detail Resep memperoleh rata-rata *Performance Score* sebesar 92, dengan nilai FCP sebesar 0,62 detik, LCP sebesar 0,72 detik, TBT sebesar 74 ms, CLS sebesar 0,0012, dan *Speed Index* sebesar 1,36. Hasil ini menunjukkan bahwa proses *rendering* di sisi klien membutuhkan waktu lebih lama karena browser harus memproses file JavaScript terlebih dahulu sebelum seluruh isi halaman dapat ditampilkan secara lengkap kepada pengguna.

Perbandingan kedua implementasi pada halaman Detail Resep menunjukkan hasil yang lebih variatif dibandingkan halaman Home. React.js CSR mencatat FCP dan LCP yang lebih rendah, yaitu 0,62 detik dan 0,72 detik, dibandingkan Laravel SSR sebesar 0,9 detik dan 0,94 detik. Namun, Laravel SSR tetap unggul pada *Performance Score* (97,6 vs 92) dan TBT (0 ms vs 74 ms), yang mengindikasikan bahwa meskipun React.js CSR mampu menampilkan konten awal lebih cepat pada halaman ini, beban pemrosesan JavaScript di sisi browser tetap mempengaruhi stabilitas dan konsistensi performanya.

C. Perbandingan Rata-Rata Laravel dan React

Berdasarkan hasil pengujian pada halaman Home dan Detail Resep, Laravel SSR menunjukkan performa yang lebih baik dibandingkan React.js CSR pada hampir seluruh metrik pengujian Lighthouse yang digunakan. Laravel SSR memperoleh rata-rata *Performance Score* sebesar 97, sedangkan React.js CSR memperoleh rata-rata 90,6.

Pada metrik FCP dan LCP, Laravel SSR menunjukkan keunggulan pada halaman Home, sementara React.js CSR mencatat waktu yang lebih rendah pada halaman Detail. Namun secara keseluruhan, Laravel SSR memperoleh *Performance Score* dan TBT yang lebih baik di kedua halaman, dengan nilai TBT konsisten di angka 0 ms dibandingkan React.js CSR yang mencapai rata-rata 198 ms pada halaman Home dan 74 ms pada halaman Detail. Hal ini menunjukkan bahwa halaman berbasis SSR dapat menampilkan konten lebih cepat karena proses *rendering* dilakukan di sisi *server* sebelum dikirim ke browser.

Selain itu, Laravel SSR memperoleh nilai TBT dan *Speed Index* yang lebih baik dibandingkan React.js CSR. Sementara itu, kedua implementasi memiliki nilai CLS yang sama-sama rendah sehingga kestabilan tampilan halaman tetap terjaga.

D. Kesimpulan

Hasil pengujian menunjukkan bahwa metode *rendering* berpengaruh terhadap performa awal halaman. Pada penelitian ini, Laravel SSR mampu memberikan waktu muat yang lebih cepat karena proses pembentukan halaman dilakukan di *server*. Hal ini membuat browser dapat menampilkan konten lebih cepat.

React.js CSR menunjukkan performa yang lebih rendah pada *Performance Score* dan TBT di kedua halaman pengujian. Nilai TBT yang tinggi dan tidak konsisten mengindikasikan adanya beban pemrosesan JavaScript di sisi browser yang mempengaruhi stabilitas performa, meskipun pada halaman Detail React.js CSR mampu mencatat FCP dan LCP yang lebih rendah dibandingkan Laravel SSR.

Dengan demikian, untuk *website* berbasis konten sederhana seperti resep masakan, Laravel SSR lebih direkomendasikan dari sisi performa pemuatan awal halaman.

REFERENCES

- [1] V. Jain, "WEB VITALS AND CORE METRICS FOR WEB PERFORMANCE OPTIMIZATION," *International Journal of Core Engineering & Management*, no. 7, p. 6, 2023.
- [2] J. Lokhande, S. Mantri, and Y. Hande, "A Comparative Analysis of Client Side Rendering and Server Side Rendering." [Online]. Available: <https://ssrn.com/abstract=5832362>
- [3] N. Wehner, M. Amir, M. Seufert, R. Schatz, and T. Hobfeld, "A Vital Improvement? Relating Google's Core Web Vitals to Actual Web QoE," in 2022 14th International Conference on Quality of Multimedia Experience, QoMEX 2022, Institute of Electrical and Electronics Engineers Inc., 2022. doi: 10.1109/QoMEX55416.2022.9900881.
- [4] J. van Riet, I. Malavolta, and T. A. Ghaleb, "Optimize along the way: An industrial case study on web performance," *Journal of Systems and Software*, vol. 198, p. 111593, Apr. 2023, doi: 10.1016/J.JSS.2022.111593.
- [5] N. Wehner, M. Seufert, R. Schatz, and T. Hoßfeld, "Do you agree? Contrasting Google's Core Web Vitals and the impact of cookie consent banners with actual web QoE," *Qual. User Exp.*, vol. 8, no. 1, Dec. 2023, doi: 10.1007/s41233-023-00058-3.
- [6] "Server-Side Rendering (SSR) - Inertia.js Documentation." Accessed: May 13, 2026. [Online]. Available: <https://inertiajs.com/docs/v3/advanced/server-side-rendering>
- [7] T. Fadhilah Iskandar, M. Lubis, T. Fabrianti Kusumasari, and A. Ridho Lubis, "Comparison between client-side and server-side rendering in the web development," in *IOP Conference Series: Materials Science and Engineering*, Institute of Physics Publishing, Jun. 2020. doi: 10.1088/1757-899X/801/1/012136.
- [8] S. Gangishetti and V. Jain, "Comparative Analysis of Client-Side vs. Server-Side Rendering for Large-Scale Content Platforms," *International Journal of AI, BigData, Computational and Management Studies*, vol. 7, no. 1, pp. 74–80, 2026, doi: 10.63282/3050-9416.IJAIBDCMS-V7I1P112.
- [9] "Rendering: Client-side Rendering (CSR) | Next.js." Accessed: May 13, 2026. [Online]. Available: <https://nextjs.org/docs/pages/building-your-application/rendering/client-side-rendering>
- [10] M. Fahrudin, T. F. Kusumasari, and E. N. Alam, "A Performance Comparison of Frontend Rendering Strategies in Laravel Applications," *Institute of Electrical and Electronics Engineers (IEEE)*, Nov. 2025, pp. 1–6. doi: 10.1109/iceeie66203.2025.11252452.
- [11] T. McGill, O. Bamgboye, X. Liu, and C. S. Kalutharage, "Towards Improving Accessibility of Web Auditing with Google Lighthouse," in *Proceedings - International Computer Software and Applications Conference*, IEEE Computer Society, 2023, pp. 1594–1599. doi: 10.1109/COMPSAC57700.2023.00246.